

Data Download System for a Satellite Fleet

Context

In Earth Observation (EO) missions, low-Earth orbit (LEO) satellites capture massive amounts of high-resolution imagery and sensor data. Because these satellites move at high velocities, they can only transmit data during brief “pass windows”—fleeting intervals when they have a direct line-of-sight to a Ground Station (GS).

Your objective is to design and implement a prototype simulation for a **Mission Control Downlink Scheduler**. The system must manage high-volume data transfers from a fleet of satellites to a single ground station. The primary challenge is maximizing the retrieval of high-priority files: the ground station can only track one satellite at a time, and the visibility windows are limited.

Note: This is an architectural and behavioral simulation; you do not need to transfer real files over a network, but rather simulate the data transmission mechanics and state changes accurately.

Fundamental Requirements

Process Architecture

The system must be implemented using **Python** and **Docker**. You may initialize the system using the following multi-process scheme:

- **1 GroundStation Process:** Acts as the scheduler and “data receiver”. It maintains the “Pass Schedule” and decides which satellite to link with at any given time.
- **2 Satellite Processes:** They simulate being two independent satellites (`satellite_1`, `satellite_2`) that “transmit” files.

Inter-Process Communication (IPC)

You must choose and justify a suitable **IPC** mechanism for communication between the ground station and the satellite processes (e.g., **Sockets**, **Redis Pub/Sub**, **gRPC**, or **Python Multiprocessing Queues/Pipes**).

Data Source Loading

- **Pass Schedule:** A list of time windows where specific satellites are visible to the GS.

```
{
  "pass_1": {
    "satellite_id": "satellite_1",
    "start": 0,
    "end": 10,
    "bandwidth_gbps": 0.5
  }
}
```

- **Satellite & Ground Station Inventory:** Each satellite and ground station process should access an independent database containing its file list: File ID, Priority (1-5, where 1 is highest), Chunk ID, Chunk Size (MB), and Status (Pending/Completed/Deleted).

Business Logic and Constraints

Downlink Optimization

- Pass windows in the schedule **can overlap** in time. Because the GS can only connect to one satellite at a time, the scheduling algorithm must dynamically decide which satellite to track at any given pass

to maximize the priority-weighted volume of data downloaded.

File Chunking and Transmission Resumption

- Files are fragmented into uniform 100MB chunks (e.g., a 1GB file contains 10 chunks).
- Transfer speed is governed by the `bandwidth_gbps` (GB/s) defined in the Pass Schedule.
- If a file successfully completes (all chunks received by the GS), the satellite must update its local database to reflect its removal or deletion.

Reporting

Upon processing the entire schedule, the GS must export a final **JSON** report detailing:

- Total GB downloaded per Priority level.
- List of completed and pending chunks.

Example report output:

```
{
  "pass_1": {
    "satellite_id": "satellite_1",
    "total_gb": { "3": 5.0 },
    "files": {
      "file_a": {
        "completed": [1, 2, ..., 50],
        "pending": [51, 52, ..., 100]
      }
    }
  },
  "pass_2": {
    "satellite_id": "satellite_2",
    "total_gb": { "1": 5.0 },
    "files": {
      "file_b": {
        "completed": [1, 2, ..., 50],
        "pending": []
      }
    }
  }
}
```

Illustrative Example

- Pass 1: `satellite_1` (T: 0-10, Bandwidth: 1GB/s). Total Capacity: 10GB.
- Pass 2: `satellite_2` (T: 5-15, Bandwidth: 1GB/s). Total Capacity: 10GB.
- `satellite_1`: File A (10GB, Priority 3).
- `satellite_2`: File B (5GB, Priority 1).

Optimal Strategy: The GS should connect to `satellite_1` from T:0-5, then switch to `satellite_2` at T:5 to get the higher Priority 1 file, then potentially switch back or stay with `satellite_2` depending on the remaining inventory.

Deliverables

1. The solution source code in a Git repository (e.g., **GitHub**, **GitLab**).
2. A `README.md` file with:
 1. Clear instructions on how to install dependencies and run the program (preferably using **Docker**).

2. A “**Design Decisions**” section justifying the technical choices, especially the selected **IPC** mechanism and the approach to the optimization algorithm.

Extra Points (Bonus)

These items are designed to assess your ability to think about scalability, robustness, and building a complete product. Choose to implement one or more of the following:

- **Dynamic Bandwidth:**
 - Simulate signal degradation as the satellite moves across the sky.

Hint: Model bandwidth variations using a sine wave relative to elapsed pass window time, where performance peaks at 100% capacity at the midpoint of the window (zenith) and degrades to 20% capacity at the start and end of the pass (horizon).
- **Web Dashboard:**
 - A simple **FastAPI** interface to:
 - * Send a new passes list to the running system.
 - * Visualize the “Pass Schedule” download events for each satellite.
- **Signal Loss:**
 - Implement a 5% chance of “Packet Drop” per chunk, requiring the chunk to be re-sent.

Evaluation Criteria

- **Code Quality:** Clarity, organization, adherence to **PEP 8**, and use of static typing (**type hints**).
- **Design and Architecture:** Quality of the multiprocessing solution, separation of responsibilities between processes, and choice of **IPC** mechanism.
- **Problem Resolution:** Correctness of the allocation algorithm to maximize downloads and manage constraints.
- **Technical Justification:** Depth and clarity of the arguments for the design decisions in the **README.md**.
- **Concurrency Handling:** Absence of race conditions, deadlocks, or other common problems in concurrent systems.

Annex

Pass schedule **JSON** schema:

```
{
  "$schema": "[https://json-schema.org/draft/2020-12/schema] (https://json-schema.org/draft/2020-12/sche",
  "title": "SatellitePassWindows",
  "type": "object",
  "patternProperties": {
    "^pass_\\d+$": {
      "type": "object",
      "required": [
        "satellite_id",
        "start",
        "end",
        "bandwidth_gbps"
      ],
      "properties": {
        "satellite_id": {
          "type": "string",
          "description": "The unique identifier for the satellite."
        }
      }
    }
  }
}
```

```

    "start": {
      "type": "integer",
      "minimum": 0,
      "description": "The start time of the window."
    },
    "end": {
      "type": "integer",
      "minimum": 0,
      "description": "The end time of the window."
    },
    "bandwidth_gbps": {
      "type": "number",
      "minimum": 0,
      "description": "The allocated bandwidth in Gigabits per second."
    }
  }
},
"additionalProperties": false
}

```

Reporting output **JSON** schema:

```

{
  "$schema": "[https://json-schema.org/draft/2020-12/schema](https://json-schema.org/draft/2020-12/sche",
  "title": "SatellitePassesDetailed",
  "type": "object",
  "patternProperties": {
    "^pass_\\d+$": {
      "type": "object",
      "required": ["satellite_id", "total_gb", "files"],
      "properties": {
        "satellite_id": {
          "type": "string",
          "description": "The unique identifier for the satellite assigned to this pass."
        },
        "total_gb": {
          "type": "object",
          "patternProperties": {
            "[1-5]$": {
              "type": "number",
              "minimum": 0
            }
          },
          "description": "Mapping of stringified priority levels (1-5) to total Gigabytes downloaded."
        },
        "files": {
          "type": "object",
          "additionalProperties": {
            "type": "object",
            "required": ["completed", "pending"],
            "properties": {
              "completed": {
                "type": "array",

```

```

        "items": {
            "type": "integer"
        },
        "description": "List of completed chunk or part IDs."
    },
    "pending": {
        "type": "array",
        "items": {
            "type": "integer"
        },
        "description": "List of pending chunk or part IDs."
    }
},
    "description": "Dictionary of files being transferred during this pass."
}
}
}
},
    "additionalProperties": false
}

```